

JusticeFlow

AI-Powered Police Case & Evidence Management System
Operating Systems Layer — Project Proposal

Submitted By:

Muhammad Furqan Sheikh (24K-0527)

Muhammad Abu Bakar (24K-0826)

Abdullah Muaz Abbassi (24K-0764)

Course: Operating Systems (CL2006 / CS2006)

1. Introduction / Background

1.1 Background

Law enforcement agencies in Pakistan continue to manage police cases, evidence, and officer operations through manual paper-based processes or outdated standalone software systems. These systems suffer from data fragmentation, lack of real-time visibility, poor chain-of-custody tracking for evidence, and no automation for time-critical legal deadlines such as warrant expiry and bail record management.

JusticeFlow is designed to address these challenges by providing a unified, AI-powered management system built on modern software and systems engineering principles, ensuring that every design decision reflects the actual operational requirements of Pakistani law enforcement.

1.2 Project Description

JusticeFlow is a multi-course integrated academic project that simultaneously satisfies the requirements of four concurrent courses: Operating Systems, Database Management Systems, Software Design and Architecture, and Artificial Intelligence. The system manages the complete lifecycle of a police case from First Information Report (FIR) registration through investigation, evidence collection, warrant issuance, arrest, forensic analysis, charge sheet submission, and court proceedings.

The core principle of the project architecture is that each course layer is load-bearing. Removing any single layer causes the entire system to fail. This is by design — it demonstrates genuine cross-disciplinary integration rather than surface-level parallelism.

1.3 The Role of the Operating Systems Layer

The OS layer is the execution environment that binds all other layers together. Without it, the three AI models never execute on schedule, the C++ application cannot communicate with PostgreSQL, concurrent officer sessions corrupt shared program state, and expired warrants and bail records are never automatically updated. The OS layer is not a wrapper or an add-on — it is the runtime foundation of the entire system.

OS Layer Core Contribution

Process management: Spawns and supervises AI agent child processes.
IPC: Connects the C++ application to PostgreSQL and AI agents.
Job scheduler: Enforces time-critical legal deadlines automatically.
Multithreading: Handles concurrent officer sessions without data corruption.
Memory management: Efficiently handles large evidence files via `mmap()`.

2. Problem Statement

The specific operating systems problems that JusticeFlow addresses are as follows:

- How to safely spawn and supervise Python AI agent processes from a C++ application without creating zombie processes or resource leaks.
- How to enable reliable communication between the C++ application, the PostgreSQL database engine, and Python AI agents using appropriate IPC mechanisms for each communication pattern.
- How to implement a daemon process that automatically enforces time-based legal deadlines — specifically warrant expiry and bail record expiry — without relying on external cron infrastructure.
- How to handle concurrent officer sessions using POSIX threads while protecting shared data structures from race conditions, deadlocks, and data corruption.
- How to efficiently access arbitrarily large binary evidence files — CCTV footage, forensic images, scanned documents — without loading them entirely into heap memory.

These are not isolated software engineering problems. They arise directly from the legal and operational requirements of a real police case management system. Warrant expiry is a legal obligation — a warrant past its valid date must be automatically invalidated. Evidence file access must be efficient because forensic processing of large files is a routine operation. Concurrent sessions are unavoidable because multiple officers at the same station work simultaneously.

3. Objectives of the Project

The primary objectives of the JusticeFlow Operating Systems layer are:

- Implement a POSIX-compliant C++ application that manages its own process lifecycle including child process creation, supervision, and cleanup using `fork()`, `exec()`, and `waitpid()`.
- Implement three distinct IPC mechanisms — Unix domain sockets for database communication, named pipes for AI agent status reporting, and POSIX shared memory for real-time status sharing — each selected for a specific communication pattern.
- Build a custom daemon process using the double-fork pattern that uses `SIGALRM` and `setitimer()` to schedule legal expiry jobs and AI model runs at configurable intervals without any external dependency.
- Implement a POSIX thread pool with `pthread_mutex_t`, `sem_t` (counting semaphore), and `pthread_cond_t` (condition variable) synchronization primitives for safe concurrent officer session handling.
- Implement memory-mapped file access using `mmap()` with `MAP_PRIVATE` and `PROT_READ` for large evidence files, and `mlock()` to pin security-critical data structures in physical RAM.
- Demonstrate comprehensive coverage of Operating Systems concepts from Silberschatz Chapters 3 through 9 through concrete, working implementations integrated into a real system.
- Produce a fully integrated OS layer that works seamlessly with the DBMS layer (PostgreSQL), the AI layer (Python agents), and the SDA layer (C++ class hierarchy) as a unified multi-course deliverable.

4. Scope of the Project

4.1 In Scope

The following are within the scope of the JusticeFlow OS layer deliverable:

- C++ application process management using POSIX APIs on Ubuntu Linux 24.04 LTS
- Three IPC mechanisms: Unix domain socket to PostgreSQL, named FIFOs for AI agents, POSIX shared memory for status table
- Custom daemon process with timer-based job scheduling for legal record expiry and AI agent invocation
- POSIX thread pool implementation for concurrent officer session handling with full synchronization
- Memory-mapped I/O for large evidence file processing
- Integration with three Python AI agents (DBSCAN, Random Forest, Hungarian Algorithm) via process spawning
- Integration with PostgreSQL database via Unix domain socket and libpq
- Performance measurement — latency, throughput, and resource utilization reporting

4.2 Out of Scope

The following are explicitly outside the scope of this deliverable:

- Custom Linux kernel module or system call implementation
- Network-facing TCP/IP socket communication — all communication is local Unix sockets
- GUI or web-based front-end — the application is a command-line C++ process
- Deployment to cloud infrastructure — the system runs on a single local Linux machine
- The AI model training and prediction logic — that is owned by each subsystem team member
- Database schema design and stored procedures — that is the DBMS course deliverable

4.3 Boundaries

Boundary	Description
Platform	Ubuntu Linux 24.04 LTS — single machine deployment
Language	C++ (POSIX APIs) for application layer; Python 3.12 for AI agents
Database	PostgreSQL 16.x — local Unix socket connection only
Concurrency	Maximum 50 concurrent officer sessions via thread pool
AI Agents	Three agents: DBSCAN (6-hourly), Random Forest (12-hourly), Hungarian (6-hourly)
Legal Jobs	Two daily jobs: <code>expire_warrants()</code> and <code>expire_bail_records()</code> at 00:01

5. Literature Review / Related Work

5.1 Police Information Management Systems

Several police information management systems have been deployed internationally. The FBI's CJIS (Criminal Justice Information Services) system and the UK's HOLMES 2 (Home Office Large Major Enquiry System) represent mature examples of large-scale law enforcement databases. Both systems share common architectural characteristics: centralized case management, strict audit trails, role-based access control, and integration with forensic databases. JusticeFlow draws from these models but adapts them to the constraints and legal framework of Pakistani law enforcement.

5.2 POSIX Process and IPC Models

The POSIX standard (IEEE Std 1003.1) provides the foundational APIs for process management and IPC on Unix-like systems. Stevens and Rago in *Advanced Programming in the UNIX Environment* (3rd Edition) provide comprehensive coverage of fork-exec patterns, signal handling, and daemon process creation. The double-fork daemon initialization pattern used in JusticeFlow is directly derived from their canonical implementation. The IPC mechanism selection — Unix sockets for database, FIFOs for one-directional status, shared memory for real-time state — follows the design guidelines outlined in Stevens' *UNIX Network Programming Volume 2*.

5.3 Multithreaded Server Design

The thread pool pattern used by JusticeFlow for concurrent officer session handling is based on the Producer-Consumer model described in Silberschatz, Galvin, and Gagne (*Operating System Concepts*, 10th Edition, Chapter 4). The semaphore-based connection limiting follows the bounded buffer problem solution. The condition variable usage for AI result notification follows the monitor-based synchronization patterns described in Chapter 6. Valgrind's Helgrind tool, used for race condition detection in the implementation, is documented in the Valgrind manual (valgrind.org).

5.4 Memory-Mapped I/O

The `mmap()` interface for file-backed virtual memory is documented in the Linux man-pages project and analyzed in depth in Love's *Linux System Programming* (2nd Edition, O'Reilly). The `MAP_PRIVATE` copy-on-write semantics used for evidence file processing, and the `mlock()` usage for pinning security-critical structures, follow the patterns described in Chapter 4 of Love's work. The performance advantages of `mmap()` over `read()` for large sequential file access are documented in Bovet and Cesati's *Understanding the Linux Kernel* (3rd Edition).

5.5 Daemon and Job Scheduling

The `SIGALRM` and `setitimer()` based scheduling approach used in JusticeFlow is a lightweight alternative to full cron infrastructure. This pattern is described in Kerrisk's *The Linux Programming Interface* (Chapter 23 — Timers and Sleeping). The double-fork daemon pattern, session creation via `setsid()`, and file descriptor cleanup are all canonical techniques documented in Kerrisk Chapter 37. The use of `WNOHANG` with `waitpid()` for non-blocking child reaping follows the signal-safe practices outlined in Chapter 26.

6. Proposed System / Methodology

6.1 Overall Approach

The OS layer will be implemented in C++ using POSIX APIs on Ubuntu Linux 24.04 LTS. Each OS component is implemented as a self-contained module within the application, following the subsystem architecture defined in the Software Design and Architecture course deliverable. The implementation follows a bottom-up build order: foundational components (process management, IPC) are built first, followed by higher-level components (scheduler, threading) that depend on them, and finally the integration layer that connects all components.

6.2 Component 1 — Process Management

The application manages a hierarchy of processes using the standard Unix fork-exec model. The main process forks a scheduler daemon using the double-fork pattern to detach it from the terminal and create a new session. The scheduler in turn spawns AI agent processes using `fork()` followed by `execl()` to replace the child image with a Python interpreter running the appropriate agent script.

A `SIGCHLD` signal handler is registered to asynchronously reap completed child processes using `waitpid()` with the `WNOHANG` flag. This prevents zombie process accumulation. A process registry maps each child PID to its agent type, enabling the scheduler to restart failed agents and log completion status to the database.

POSIX API	Purpose
<code>fork()</code>	Create child process for scheduler daemon and AI agents
<code>execl()</code>	Replace child image with Python AI agent interpreter
<code>waitpid(WNOHANG)</code>	Non-blocking child reaping — zombie process prevention
<code>setsid()</code>	Create new session for daemon process detachment
<code>sigaction(SIGCHLD)</code>	Register asynchronous child termination handler
<code>getpid()</code> / <code>getppid()</code>	Process identity for logging and audit trail

6.3 Component 2 — Inter-Process Communication

Three IPC mechanisms are used, each selected based on the communication requirements of the specific channel it serves.

Unix Domain Socket — Application to PostgreSQL

The C++ application connects to the PostgreSQL database engine using a Unix domain socket rather than a TCP/IP socket. This is both faster (no network stack overhead) and more secure (file system permissions control access). PostgreSQL natively listens on `/var/run/postgresql/.s.PGSQL.5432`. The `libpq` library is used on the C++ side to manage the socket connection.

Named Pipes (FIFOs) — AI Agents to Scheduler

Each AI agent communicates its execution status back to the scheduler through a dedicated named pipe created with `mkfifo()`. When an agent completes its analysis run, it writes a status message containing completion time, number of predictions generated, and error codes to its named pipe. The scheduler reads from these pipes asynchronously to maintain an accurate view of agent health without polling the database.

POSIX Shared Memory — Scheduler to Application

A shared memory segment created with `shm_open()` and mapped with `mmap()` holds a real-time agent status table that both the scheduler and the main application can read. This allows the application to display live AI agent status on the officer dashboard without a database query for every status check. Access to the shared memory segment is protected by a POSIX mutex stored within the segment itself.

IPC Mechanism	Between	Direction	Reason for Choice
Unix domain socket	C++ app ↔ PostgreSQL	Bidirectional	Fastest local socket; natively supported by PostgreSQL
Named pipe (FIFO)	AI agents → Scheduler	One-directional	Simple, lightweight; agents only need to report status
POSIX shared memory	Scheduler ↔ Application	Bidirectional	Zero-copy; eliminates DB query for status dashboard

6.4 Component 3 — Job Scheduler Daemon

A dedicated daemon process handles all time-based operations. It is initialized using the standard double-fork pattern: fork once to orphan the process, call `setsid()` to create a new session with no controlling terminal, fork again to prevent terminal reacquisition, then close all inherited file descriptors and redirect standard streams to `/dev/null`.

Once running as a true daemon, the scheduler uses `SIGALRM` with `setitimer()` to fire a signal every 60 seconds. A tick counter inside the signal handler drives all scheduled jobs at their respective intervals. Legal expiry jobs are executed as direct PostgreSQL function calls via the `libpq` connection. AI agent runs are executed by forking a child and replacing it with the appropriate Python script.

Scheduled Job	Interval	Mechanism	Database Function
Expire warrants	Daily at 00:01	Direct SQL call	<code>expire_warrants()</code>
Expire bail records	Daily at 00:01	Direct SQL call	<code>expire_bail_records()</code>
Expire AI suggestions	Every hour	Direct SQL call	<code>expire_workload_assignments()</code>
Run DBSCAN hotspot agent	Every 6 hours	fork + exec	<code>hotspot_agent.py</code>

Scheduled Job	Interval	Mechanism	Database Function
Run Random Forest agent	Every 12 hours	fork + exec	priority_agent.py
Run Hungarian agent	Every 6 hours	fork + exec	workload_agent.py
Database health check	Every hour	PQstatus() check	— (reconnect if needed)

6.5 Component 4 — Multithreading

The main application uses a POSIX thread pool to handle concurrent officer sessions. When an officer connects, the main thread checks the connection semaphore. If the maximum concurrent session limit has not been reached, it decrements the semaphore and spawns a new pthread to handle that session. The worker thread authenticates the officer, sets the PostgreSQL session variable, processes requests, and releases the semaphore slot upon disconnection.

Shared data structures — the session registry and connection pool — are protected by pthread_mutex_t. A condition variable (pthread_cond_t) is used to notify worker threads when fresh AI agent results are available in shared memory, avoiding busy-waiting.

Synchronization Primitive	Purpose	Where Used
pthread_mutex_t	Mutual exclusion on shared data	Session registry, connection pool
sem_t (counting)	Limit maximum concurrent sessions	Connection acceptor loop
pthread_cond_t	Notify threads of AI result availability	Dashboard data refresh
pthread_rwlock_t	Read-heavy configuration data	System configuration table

6.6 Component 5 — Memory Management

Evidence files — including CCTV footage, digital forensics images, and scanned documents — can be arbitrarily large and cannot be loaded into heap memory entirely. The application uses mmap() to memory-map these files into its virtual address space with MAP_PRIVATE and PROT_READ flags. The kernel then handles paging automatically, loading only the pages that are actually accessed. When processing is complete, munmap() releases the mapping cleanly.

The session table and connection pool are pinned in physical RAM using mlock() to guarantee they are never swapped to disk. This is critical for security — session tokens and officer credentials must not appear in swap space — and for performance, as these structures are accessed on every incoming request.

API	Usage	Purpose
mmap(MAP_PRIVATE)	Evidence file access	Virtual memory mapping — kernel handles paging on demand
munmap()	After evidence processing	Release virtual address space mapping
mlock()	Session table, connection pool	Pin in physical RAM — no swap, security critical
msync()	Shared memory updates	Flush mmap'd writes to backing store

7. System Architecture / Design

7.1 Process Topology

The JusticeFlow runtime consists of the following processes and threads running concurrently on the Linux host:

Process / Thread	Type	Role
justiceflow_main	C++ application process	Main entry point, connection acceptor, request router
Worker threads	POSIX pthreads (inside main)	Handle individual officer sessions concurrently
justiceflow_scheduler	Daemon process (forked from main)	Runs scheduled jobs, spawns AI agents on schedule
hotspot_agent.py	Child process (fork + exec)	DBSCAN crime hotspot analyzer — Subsystem 1 AI
priority_agent.py	Child process (fork + exec)	Random Forest case priority recommender — Subsystem 2 AI
workload_agent.py	Child process (fork + exec)	Hungarian Algorithm workload balancer — Subsystem 3 AI
PostgreSQL	External process	Database engine, listens on Unix domain socket

7.2 Layered Architecture

The system is organized into four horizontal layers, each with a distinct responsibility:

Layer	Components	Responsibility
Application Layer	Subsystem 1, 2, 3 (C++ modules)	Business logic — case management, investigation, enforcement
Infrastructure Layer	Authentication & Session module	Identity and access control — shared by all subsystems
Data Layer	PostgreSQL (public, audit, analytics)	Persistence, constraints, triggers, audit trail
OS Layer	Process Mgmt, IPC, Scheduler, Threads, Memory	Runtime execution environment for all layers

7.3 IPC Communication Diagram

The following describes the communication channels between all system components:

- justiceflow_main $\longleftrightarrow$ PostgreSQL via Unix domain socket (libpq)
- justiceflow_scheduler $\rightarrow$ PostgreSQL via Unix domain socket (direct SQL calls for expiry jobs)
- justiceflow_scheduler $\rightarrow$ AI agents via fork() + exec() (process spawning)
- AI agents $\rightarrow$ justiceflow_scheduler via named FIFOs (status reporting)
- justiceflow_scheduler $\longleftrightarrow$ justiceflow_main via POSIX shared memory (agent status table)
- Worker threads $\longleftrightarrow$ shared data structures via pthread_mutex (session registry, connection pool)

8. Operating System Concepts Used

The following table maps each major OS concept from the course syllabus (Silberschatz Operating System Concepts) to its concrete JusticeFlow implementation:

OS Concept	Textbook Chapter	JusticeFlow Implementation
Process creation and termination	Chapter 3	fork(), execl(), waitpid() for AI agent lifecycle management
Orphan and zombie process prevention	Chapter 3	SIGCHLD handler with waitpid(WNOHANG) — non-blocking reap
Daemon processes	Chapter 3	Double-fork pattern — justiceflow_scheduler daemon
Signals	Chapter 3	SIGALRM for timer-based scheduling, SIGCHLD for child cleanup
Pipes	Chapter 3	Named FIFOs (mkfifo) for AI agent status reporting to scheduler
Shared memory	Chapter 3	shm_open() + mmap() for real-time agent status table
Sockets (IPC)	Chapter 3	AF_UNIX domain socket connecting application to PostgreSQL
Threads	Chapter 4	pthread_create() — thread pool for concurrent officer sessions
Mutex	Chapter 5	pthread_mutex_t — protecting session registry and connection pool
Semaphores	Chapter 5	sem_t (counting) — enforcing maximum concurrent connection limit
Condition variables	Chapter 5	pthread_cond_t — notifying threads of fresh AI results
Read-write locks	Chapter 5	pthread_rwlock_t — efficient read-heavy config data access
Process synchronization	Chapter 5	Full synchronization suite preventing race conditions
Deadlock prevention	Chapter 7	Consistent mutex lock ordering across all critical sections
Virtual memory	Chapter 9	mmap() — mapping large evidence files into virtual address space
Demand paging	Chapter 9	MAP_PRIVATE — kernel pages evidence files on demand
Physical memory locking	Chapter 9	mlock() — pins session table and credentials in physical RAM
CPU scheduling	Chapter 5	SCHED_OTHER for worker threads, elevated priority for timer thread

9. Tools and Technologies

Tool / Technology	Version	Role in Project
Ubuntu Linux	24.04 LTS	Host operating system for development and deployment
C++ Compiler	GCC 13 (g++)	Compiling the main application and all C++ modules
POSIX APIs	IEEE Std 1003.1	pthread, IPC primitives, signals, virtual memory management
PostgreSQL	16.x	Relational database engine — connected via Unix domain socket
libpq	16.x	Official C library for PostgreSQL connection from C++ application
Python	3.12	Runtime environment for all three AI agent scripts
psycopg2	2.9.x	PostgreSQL adapter for Python — used by all three AI agents and Flask API
NumPy	1.26.x	Numerical computation — cost matrix construction for Hungarian AI
SciPy	1.12.x	Hungarian Algorithm solver via <code>scipy.optimize.linear_sum_assignment</code>
scikit-learn	1.4.x	DBSCAN and Random Forest model implementations
GDB	14.x	Debugging multi-process and multi-threaded C++ code
Valgrind (Helgrind)	3.21	Detecting memory leaks and thread race conditions
draw.io (diagrams.net)	Latest	UML diagram creation for SDA deliverable
Git	2.43	Version control — shared repository across all team members
Flask	3.x	Lightweight Python web framework — serves dashboard API and HTML
Chart.js	4.x	JavaScript charting library — renders live charts in browser
VS Code	Latest	Primary IDE with C++ and Python extensions

9.1 Visualization Interface

The system provides *two* interface layers. The primary operational interface is a C++ command-line application used by officers for all case management operations. A supplementary web-based dashboard is implemented using Flask and Chart.js to satisfy GUI visualization requirements and provide analytical insight for SHOs.

The **Flask** dashboard is a read-only Python web application that connects to PostgreSQL using **psycopg2** and exposes a **REST API** consumed by **Chart.js** in the browser. It runs as an independent process and has no write access to any database schema.

Six dashboard tabs are provided covering OS layer metrics (active officer sessions, thread pool utilization), AI agent status (last run time, predictions generated, model accuracy trend), crime hotspot visualization (DBSCAN cluster map with risk level color coding), case priority distribution (Random Forest scores by priority level), workload assignment suggestions (Hungarian Algorithm pending decisions), and application analytics (cases filed per month, evidence by type, warrant and bail status breakdowns).

Real-time Matplotlib windows are additionally available for developer-level monitoring — specifically a live session counter updated from the POSIX shared memory segment and an AI agent health monitor updated via the named pipe IPC channel.

10. Implementation Plan & Timeline

10.1 Build Order

The OS layer is built bottom-up. Each phase unlocks the next:

- Phase 1: IPC foundation — Unix socket to PostgreSQL must exist before any other component can communicate with the database.
- Phase 2: Process management and daemon — the scheduler must exist before AI agents can be spawned on schedule.
- Phase 3: Threading — the thread pool can be built in parallel with Phase 2 since it only depends on the database connection.
- Phase 4: Memory management — mmap() evidence handler is independent and can be built any time after Phase 1.
- Phase 5: Integration — all components are connected and tested as a unified system.

10.2 Weekly Timeline

Week	Deliverable	OS Components Covered
Week 1	Unix domain socket connection to PostgreSQL established and tested	IPC — AF_UNIX sockets, libpq connection
Week 1	Auth & Session C++ layer — officer login and session variable injection	Process identity, session context
Week 2	POSIX thread pool implemented — pthread_create, mutex, semaphore, condvar	Threads, synchronization primitives
Week 2	Daemon initialization — double fork, setsid(), signal handler registration	Daemon processes, signals, setsid()
Week 3	Job scheduler operational — SIGALRM, setitimer(), daily and hourly jobs	SIGALRM, setitimer, tick-based scheduling
Week 3	Named pipes and shared memory IPC for AI agent communication	FIFOs, shm_open(), mmap(), mutex
Week 4	mmap() evidence file handler — MAP_PRIVATE, mlock() for session table	Virtual memory, demand paging, mlock()
Week 4	AI agent fork-exec spawning fully integrated with scheduler	fork, execl, SIGCHLD, process registry
Week 5	Full integration testing — all 5 components running together	End-to-end system validation
Week 5	Performance measurement — latency, throughput, resource utilization	Profiling with GDB, Valgrind, /proc/PID

10.3 Team Responsibilities

Member	OS Responsibilities
Muhammad Furqan Sheikh (24K-0527)	Auth & Session C++ layer, Job scheduler daemon, Hungarian AI agent process management, IPC for Subsystem 3
Muhammad Abu Bakar (24K-0827)	FIR and Evidence C++ modules, POSIX thread pool implementation, mmap() evidence file handler, IPC for Subsystem 2
Abdullah Muaz Abbassi (24K-0764)	Officer & Personnel C++ layer, DBSCAN and Random Forest agent spawning, Unix socket connection management, IPC for Subsystem 1

10.4 Test Plan

A formal test case document covering all four course layers has been prepared separately. It contains 57 test cases organized across 19 modules with three test cases per module. Each test case specifies a Test ID, test type, objective, precondition, input or action, and expected result. The Actual Result and Pass/Fail columns are filled by the responsible team member during implementation.

The OS layer test cases specifically cover the following categories:

Concurrency

TC-OS-10 verifies that 50 simultaneous officer sessions are handled without crash or data corruption.

TC-OS-11 verifies the semaphore correctly blocks the 51st connection until a slot is freed.

TC-OS-12 uses Valgrind Helgrind to confirm zero race conditions on the shared session registry under 20 concurrent threads.

IPC

TC-OS-04 verifies the Unix domain socket connection to PostgreSQL.

TC-OS-05 verifies correct named pipe message passing from AI agent to scheduler.

TC-OS-06 verifies automatic reconnection after PostgreSQL restart.

Memory Management

TC-OS-13 verifies mmap() demand paging behavior on a 1GB evidence file by checking that VmRSS increases by less than 10MB.

TC-OS-14 verifies mlock() prevents session table pages from appearing in swap space under memory pressure.

TC-OS-15 uses Valgrind memcheck to confirm zero bytes leaked after 100 officer session cycles.

Process Management

TC-OS-07 verifies daemon detachment by checking the TTY column in ps output.

TC-OS-08 verifies SIGALRM fires at the correct interval.

TC-OS-09 verifies the daily expiry job runs exactly once at midnight over a 24-hour run.

11. Expected Results and Conclusion

11.1 Expected Results

Upon successful completion of the JusticeFlow Operating Systems layer, the following outcomes are expected:

Functional Outcomes

- A running C++ application that manages its own process hierarchy — spawning, supervising, and cleaning up AI agent child processes without zombie accumulation.
- Three working IPC channels — Unix domain socket to PostgreSQL, named FIFOs for AI agent status, and POSIX shared memory for the real-time status dashboard.
- A daemon scheduler that automatically expires warrants and bail records daily at 00:01 and invokes the three AI agents at their configured intervals, operating independently of any external cron service.
- A thread pool handling up to 50 concurrent officer sessions with no race conditions, deadlocks, or data corruption, verified by Valgrind Helgrind.
- Memory-mapped evidence file access confirmed to use a fraction of the heap memory compared to a read()-based implementation for the same file sizes.

Measurable Performance Targets

Metric	Target	Measurement Method
Max concurrent officer sessions	50 without degradation	Load test with 50 simultaneous connections
Warrant expiry job execution time	< 500ms for 10,000 records	pg_stat_activity timing
AI agent spawn time	< 200ms from fork() to Python startup	gettimeofday() before/after fork-exec
Evidence file mmap() access	< 50ms for first page access on 1GB file	Compared to read() baseline
Zombie processes after agent completion	Zero	ps aux grep Z after 24h run

11.2 Conclusion

JusticeFlow demonstrates that Operating Systems concepts are not abstract textbook exercises — they are the practical foundation of any non-trivial software system. Every component of the OS layer in this project exists because a real operational requirement demanded it. Warrants must expire on a schedule, so a daemon scheduler is needed. Multiple officers work simultaneously, so a thread pool is needed. Evidence files are large, so memory-mapped I/O is needed. AI models must run in the background, so process management is needed.

The five OS components together form a cohesive runtime layer that makes JusticeFlow function as a single integrated system rather than four disconnected course deliverables. The implementation will provide hands-on experience with process lifecycle management, multiple IPC mechanisms, signal-driven scheduling, POSIX thread synchronization, and virtual memory management — covering the breadth of OS concepts from Chapters 3 through 9 of the Silberschatz textbook through real, working code.

Integration Guarantee

All three team members connect to one shared PostgreSQL instance via the same Unix domain socket.

All three AI agents are spawned and supervised by the same scheduler daemon.

All three C++ subsystems share the same thread pool, mutex conventions, and IPC infrastructure.

This is one unified OS layer — not three separate parallel implementations.